

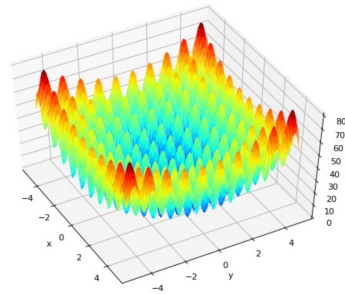
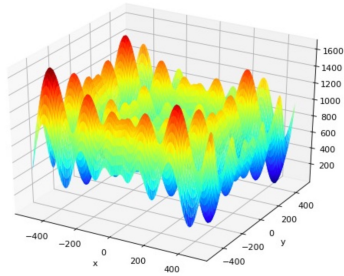
# Natural Computing

LMU Munich  
summer term 2025

Thomas Gabor



## The Solution Landscape Metaphor



Natural Computing, summer term 2025, LMU Munich

source: [deap.readthedocs.io/en/master/api/benchmarks.html](https://deap.readthedocs.io/en/master/api/benchmarks.html)

188

**Algorithm 1** (single-sample random search). Let  $\mathcal{D} = (\mathcal{X}, \mathcal{T}, \tau, e, \langle x_u \rangle_{0 \leq u \leq t})$  be an optimization process. The process  $\mathcal{D}$  continues via (single-sample) random search if  $e$  is of the form

$$e(\langle x_u \rangle_{0 \leq u \leq t}, \tau) = x_{t+1} = \arg \min_{x \in \{x_t, x'_t\}} \tau(x)$$

where  $x'_t \sim \mathcal{X}$  is drawn at random for each call to  $e$ .

**Algorithm 2** (stochastic hill climbing). Let  $\mathcal{D} = (\mathcal{X}, \mathcal{T}, \tau, e, \langle x_u \rangle_{0 \leq u \leq t})$  be an optimization process. Let  $neighbors : \mathcal{X} \rightarrow \wp(\mathcal{X})$  be a function that returns a set of neighbors of a given state  $x \in \mathcal{X}$ . The process  $\mathcal{D}$  continues via stochastic hill climbing if  $e$  is of the form

$$e(\langle x_u \rangle_{0 \leq u \leq t}, \tau) = x_{t+1} = \arg \min_{x \in \{x_t, x'_t\}} \tau(x)$$

where  $x'_t \sim neighbors(x_t)$  is drawn at random for each call to  $e$ .

**Algorithm 3** (simulated annealing). Let  $\mathcal{D} = (\mathcal{X}, \mathcal{T}, \tau, e, \langle x_u \rangle_{0 \leq u \leq t})$  be an optimization process. Let  $neighbors : \mathcal{X} \rightarrow \wp(\mathcal{X})$  be a function that returns a set of neighbors of a given state  $x \in \mathcal{X}$ . Let  $\kappa : \mathbb{N} \rightarrow \mathbb{R}$  be a temperature schedule, i.e., a function that returns a temperature value for each time step. Let  $A : \mathcal{T} \times \mathcal{T} \times \mathbb{R} \rightarrow \mathbb{P}$  with  $\mathbb{P} = [0; 1] \subset \mathbb{R}$  be a function that returns an acceptance probability given two target values and a temperature. Commonly, we use

$$A(Q, Q', K) = e^{\frac{-(Q' - Q)}{K}}$$

for  $\mathcal{T} \subseteq \mathbb{R}$  and Euler's number  $e$ . The process  $\mathcal{D}$  continues via simulated annealing if  $e$  is of the form

$$e(\langle x_u \rangle_{0 \leq u \leq t}, \tau) = x_{t+1} = \begin{cases} x'_t & \text{if } \tau(x'_t) \leq \tau(x_t) \text{ or } r \leq A(\tau(x_t), \tau(x'_t), \kappa(t)), \\ x_t & \text{otherwise,} \end{cases}$$

where  $x'_t \sim neighbors(x_t)$  and  $r \sim \mathbb{P}$  are drawn at random for each call to  $e$ .

**Algorithm 4** (gradient descent). Let  $\mathcal{D} = (\mathcal{X}, \mathcal{T}, \tau, e, \langle x_u \rangle_{0 \leq u \leq t})$  be an optimization process. Let  $\mathcal{T}$  be continuous ( $\mathcal{T} = \mathbb{R}$ , e.g.) and let  $\tau' : \mathcal{X} \rightarrow \mathcal{T}$  be the first derivative of  $\tau$ . The process  $\mathcal{D}$  continues via gradient descent (with update rate  $\alpha \in \mathbb{R}^+$ ) if  $e$  is of the form

$$e(\langle x_u \rangle_{0 \leq u \leq t}, \tau) = x_{t+1} = x_t - \alpha \cdot \tau'(x_t).$$

The learning rate  $\alpha$  can also be given as a function, usually  $\alpha : \mathbb{N} \rightarrow \mathbb{R}$  so that  $e(\langle x_u \rangle_{0 \leq u \leq t}, \tau) = x_{t+1} = x_t - \alpha(t) \cdot \tau'(x_t)$ . If  $\tau$  is stochastic, this process is called stochastic gradient descent (SGD).

**Algorithm 5** (simulated annealing with local gradient descent). For  $n \in \mathbb{N}$ ,  $\mathcal{D} = (\mathcal{X}, \mathcal{T}, \tau, e_{\mathcal{D}}, \langle x_u \rangle_{0 \leq u \leq t})$  is an optimization process that continues via simulated annealing with  $n$  steps of local gradient descent if  $e_{\mathcal{D}}$  has the form

$$e_{\mathcal{D}}(\langle x_u \rangle_{0 \leq u \leq t}, \tau) = x_{t+1} = \begin{cases} {}^*\mathcal{E}(x'_t) & \text{if } \tau({}^*\mathcal{E}(x'_t)) \leq \tau(x_t) \\ & \text{or } r \leq A(\tau(x_t), \tau({}^*\mathcal{E}(x'_t)), \kappa(t)), \\ x_t & \text{otherwise,} \end{cases}$$

where  $x'_t, r, A, \kappa$  are given as in Alg. 3 and where  $\mathcal{E}(x'_t) = (\mathcal{X}, \mathcal{T}, \tau, e_{\mathcal{E}}, \langle x'_{t+v} \rangle_{0 \leq v \leq n})$  is an optimization process that continues via gradient descent (cf. Alg. 4) so that

$${}^*\mathcal{E}(x'_t) = \arg \min_{\substack{x'_{t+v}, \\ 0 \leq v \leq n}} \tau(x'_{t+v})$$

is the best solution found by  $\mathcal{E}$  when starting at  $x'_t$ .

---

**Algorithm 1:** *Adam*, our proposed algorithm for stochastic optimization. See section 2 for details, and for a slightly more efficient (but less clear) order of computation.  $g_t^2$  indicates the elementwise square  $g_t \odot g_t$ . Good default settings for the tested machine learning problems are  $\alpha = 0.001$ ,  $\beta_1 = 0.9$ ,  $\beta_2 = 0.999$  and  $\epsilon = 10^{-8}$ . All operations on vectors are element-wise. With  $\beta_1^t$  and  $\beta_2^t$  we denote  $\beta_1$  and  $\beta_2$  to the power  $t$ .

---

**Require:**  $\alpha$ : Stepsize

**Require:**  $\beta_1, \beta_2 \in [0, 1)$ : Exponential decay rates for the moment estimates

**Require:**  $f(\theta)$ : Stochastic objective function with parameters  $\theta$

**Require:**  $\theta_0$ : Initial parameter vector

$m_0 \leftarrow 0$  (Initialize 1<sup>st</sup> moment vector)

$v_0 \leftarrow 0$  (Initialize 2<sup>nd</sup> moment vector)

$t \leftarrow 0$  (Initialize timestep)

**while**  $\theta_t$  not converged **do**

$t \leftarrow t + 1$

$g_t \leftarrow \nabla_{\theta} f_t(\theta_{t-1})$  (Get gradients w.r.t. stochastic objective at timestep  $t$ )

$m_t \leftarrow \beta_1 \cdot m_{t-1} + (1 - \beta_1) \cdot g_t$  (Update biased first moment estimate)

$v_t \leftarrow \beta_2 \cdot v_{t-1} + (1 - \beta_2) \cdot g_t^2$  (Update biased second raw moment estimate)

$\hat{m}_t \leftarrow m_t / (1 - \beta_1^t)$  (Compute bias-corrected first moment estimate)

$\hat{v}_t \leftarrow v_t / (1 - \beta_2^t)$  (Compute bias-corrected second raw moment estimate)

$\theta_t \leftarrow \theta_{t-1} - \alpha \cdot \hat{m}_t / (\sqrt{\hat{v}_t} + \epsilon)$  (Update parameters)

**end while**

**return**  $\theta_t$  (Resulting parameters)

---



How do these algorithms work?

# Local vs. Global Information

# Smoothness

# Deception

# The Exploration/Exploitation Dilemma

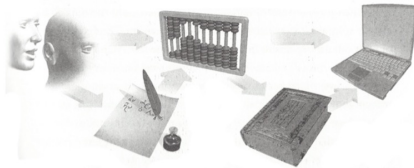
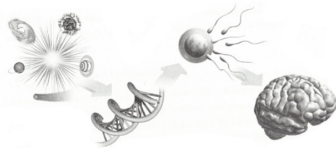
Which algorithm do we choose?

# No Free Lunch Theorem

**Theorem 3** (no free lunch [1, 3]). As measured by sample efficiency, i.e., the achieved minimal value of  $\tau$  per evaluations of  $\tau(x)$  for some new  $x \in \mathcal{X}$  for finite  $\mathcal{X}$ , all optimization algorithms perform the same when averaged over all possible target functions  $\tau$ . So, for any search/optimization algorithm, any elevated performance over one class of problems is exactly paid for in performance over another class.

# Artificial Chemistries





**“The history of the universe can be thought of as a sequence of information processing revolutions, each of which builds on the technology of the previous ones.”**

Seth Lloyd. Programming the Universe. Vintage Books, 2006.

# On Multi-Sets...

Let  $X$  be a set with members from a space  $\mathcal{X}$ .

We can identify a set  $X$  with its membership function  $\mu_X : \mathcal{X} \rightarrow \mathbb{B}$  such that

$$\mu_X(x) = 1 \iff x \in X.$$

For a multiset  $X'$  we generalize the membership function so that

$$\mu_{X'}(x) \geq 1 \iff x \in X'.$$

In both cases, the subset relation between (multi-)sets  $X, Y$  can be defined as

$$X \subseteq Y \iff \forall x \in X : \mu_X(x) \leq \mu_Y(x).$$

We then define  $\wp(X) = 2^X = \mathbb{B}^X \cong X \rightarrow \mathbb{B}$ ,

and  $\wp^*(X) = \mathbb{N}^X \cong X \rightarrow \mathbb{N}$ ,

to be the powerset functions for sets and multisets, respectively.

# More on Multi-Sets...

**Definition 10** (artificial chemistry, soup). Let  $\mathcal{X}$  be a state space. Let  $\mathcal{R}$  be a set of reaction rules  $R \in \mathcal{R}$  where each  $R$  is a function  $R : \mathcal{X}_R^{\text{dom}} \rightarrow \mathcal{X}_R^{\text{cod}}$  for  $\mathcal{X}_R^{\text{dom}}, \mathcal{X}_R^{\text{cod}} \subseteq \wp^*(\mathcal{X})$ . Let  $\mathcal{A}(X, \mathcal{R})$  be the set of all *reaction materials* from  $X \in \wp^*(\mathcal{X})$  for all rules in  $\mathcal{R}$ , i.e.,

$$\mathcal{A}(X, \mathcal{R}) = \{(X', R) \mid X' \subseteq X \wedge X' \in \mathcal{X}_R^{\text{dom}} \wedge R \in \mathcal{R}\}.$$

Let  $A : \wp^*(\mathcal{X}) \rightarrow \wp^*(\mathcal{X}) \times \mathcal{R}$  be a possibly randomized or non-deterministic function that takes a multi-set  $X \in \wp^*(\mathcal{X})$  and returns a multi-set of reactants and a reaction rule to perform so that  $A(X) \in \mathcal{A}(X, \mathcal{R})$ . The tuple  $(\mathcal{X}, \mathcal{R}, A)$  defines an artificial chemistry. A multi-set  $X \in \wp^*(\mathcal{X})$  of an artificial chemistry is also called soup; each element  $x \in X$  is also called a particle. The evolution of a soup for  $g$  generations is a sequence  $\langle X_0, \dots, X_g \rangle$  so that  $X_0$  is given as the initial soup and, for all  $t$  with  $0 \leq t \leq g - 1$

$$X_{t+1} = (X_t \setminus X') \uplus R(X')$$

where  $(X', R) = A(X_t)$ .

**Definition 10** (artificial chemistry, soup). Let  $\mathcal{X}$  be a state space. Let  $\mathcal{R}$  be a set of reaction rules  $R \in \mathcal{R}$  where each  $R$  is a function  $R : \mathcal{X}_R^{\text{dom}} \rightarrow \mathcal{X}_R^{\text{cod}}$  for  $\mathcal{X}_R^{\text{dom}}, \mathcal{X}_R^{\text{cod}} \subseteq \wp^*(\mathcal{X})$ . Let  $\mathcal{A}(X, \mathcal{R})$  be the set of all *reaction materials* from  $X \in \wp^*(\mathcal{X})$  for all rules in  $\mathcal{R}$ , i.e.,

$$\mathcal{A}(X, \mathcal{R}) = \{(X', R) \mid X' \subseteq X \wedge X' \in \mathcal{X}_R^{\text{dom}} \wedge R \in \mathcal{R}\}.$$

Let  $A : \wp^*(\mathcal{X}) \rightarrow \wp^*(\mathcal{X}) \times \mathcal{R}$  be a possibly randomized or non-deterministic function that takes a multi-set  $X \in \wp^*(\mathcal{X})$  and returns a multi-set of reactants and a reaction rule to perform so that  $A(X) \in \mathcal{A}(X, \mathcal{R})$ . The tuple  $(\mathcal{X}, \mathcal{R}, A)$  defines an artificial chemistry. A multi-set  $X \in \wp^*(\mathcal{X})$  of an artificial chemistry is also called soup; each element  $x \in X$  is also called a particle. The evolution of a soup for  $g$  generations is a sequence  $\langle X_0, \dots, X_g \rangle$  so that  $X_0$  is given as the initial soup and, for all  $t$  with  $0 \leq t \leq g - 1$

$$X_{t+1} = (X_t \setminus X') \uplus R(X')$$

where  $(X', R) = A(X_t)$ .

Reaction rules are usually written in the form  $R = \sum_i x_i \rightarrow \sum_j y_j$  for  $x_i, y_j \in \mathcal{X}$ ,  $i, j \in \mathbb{N}$ ,  $\{x_i\}_i \in \mathcal{X}_R^{\text{dom}}$ ,  $\{y_j\}_j \in \mathcal{X}_R^{\text{cod}}$ , and  $R(\{x_i\}_i) = \{y_j\}_j$ .

**Example 1** ([2]). We imagine a colony of chameleons who can either assume a red, green, or blue color. Whenever two chameleons with a different color meet, they assume the third color (that none of them has).

Let  $\mathcal{X} = \{r, g, b\}$ .

Let  $\mathcal{R} = \{R_1, R_2, R_3, R_4, R_5, R_6\}$  with

$$R_1 = r + r \rightarrow r + r,$$

$$R_2 = r + g \rightarrow b + b,$$

$$R_3 = r + b \rightarrow g + g,$$

$$R_4 = b + b \rightarrow b + b,$$

$$R_5 = b + g \rightarrow r + r,$$

$$\text{and } R_6 = g + g \rightarrow g + g.$$

Let  $A(X) \sim \mathcal{A}(X, \mathcal{R})$ .

Let  $X_0 \sim \wp^*(\mathcal{X})$ .

# Experiment Time!

**Example 1** ([2]). We imagine a colony of chameleons who can either assume a red, green, or blue color. Whenever two chameleons with a different color meet, they assume the third color (that none of them has).

Let  $\mathcal{X} = \{r, g, b\}$ .

Let  $\mathcal{R} = \{R_1, R_2, R_3, R_4, R_5, R_6\}$  with

$$R_1 = r + r \rightarrow r + r,$$

$$R_2 = r + g \rightarrow b + b,$$

$$R_3 = r + b \rightarrow g + g,$$

$$R_4 = b + b \rightarrow b + b,$$

$$R_5 = b + g \rightarrow r + r,$$

$$\text{and } R_6 = g + g \rightarrow g + g.$$

Let  $A(X) \sim \mathcal{A}(X, \mathcal{R})$ .

Let  $X_0 \sim \wp^*(\mathcal{X})$ .

early results

|  |  |  |  |
|--|--|--|--|
|  |  |  |  |
|--|--|--|--|

|  |  |  |  |
|--|--|--|--|
|  |  |  |  |
|--|--|--|--|

|  |  |  |  |
|--|--|--|--|
|  |  |  |  |
|--|--|--|--|



late results

|  |  |  |  |
|--|--|--|--|
|  |  |  |  |
|--|--|--|--|

|  |  |  |  |
|--|--|--|--|
|  |  |  |  |
|--|--|--|--|

|  |  |  |  |
|--|--|--|--|
|  |  |  |  |
|--|--|--|--|

# Experiment Time! *again*

**Example 1** ([2]). We imagine a colony of chameleons who can either assume a red, green, or blue color. Whenever two chameleons with a different color meet, they assume the third color (that none of them has).

Let  $\mathcal{X} = \{r, g, b\}$ .

Let  $\mathcal{R} = \{R_1, R_2, R_3, R_4, R_5, R_6\}$  with

$$R_1 = r + r \rightarrow r + r,$$

$$R_2 = r + g \rightarrow b + b,$$

$$R_3 = r + b \rightarrow g + g,$$

$$R_4 = b + b \rightarrow b + b,$$

$$R_5 = b + g \rightarrow r + r,$$

$$\text{and } R_6 = g + g \rightarrow g + g.$$

Let  $A(X) \sim \mathcal{A}(X, \mathcal{R})$ .

Let  $X_0 \sim \wp^*(\{g, b\})$ .

early results

|  |  |  |  |
|--|--|--|--|
|  |  |  |  |
|--|--|--|--|

|  |  |  |  |
|--|--|--|--|
|  |  |  |  |
|--|--|--|--|

|  |  |  |  |
|--|--|--|--|
|  |  |  |  |
|--|--|--|--|

late results

|  |  |  |  |
|--|--|--|--|
|  |  |  |  |
|--|--|--|--|

|  |  |  |  |
|--|--|--|--|
|  |  |  |  |
|--|--|--|--|

|  |  |  |  |
|--|--|--|--|
|  |  |  |  |
|--|--|--|--|

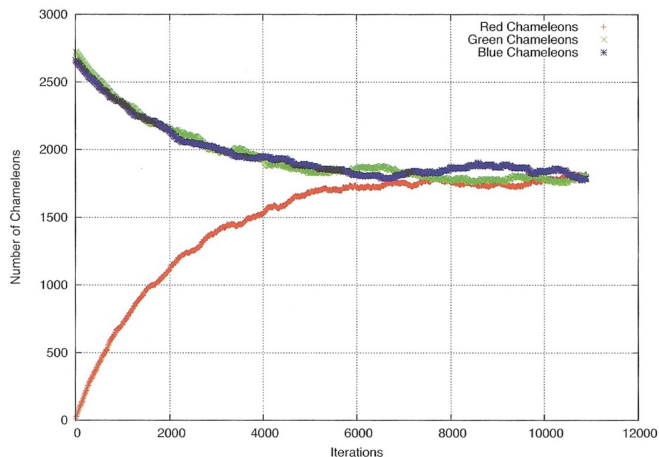


Figure 2.5 The first two generations (equivalent to 10,800 iterations) of an explicit simulation of encounters between chameleons changing colors. A total of 5,400 chameleons is simulated, starting from a condition where only blue and read chameleons are present, in virtually equal numbers, due to randomized initial condition. Red chameleons quickly pick up in numbers, until only fluctuations are left between numbers.

**Example 2** ([2]). A soup of natural numbers divides each other...?

Let  $\mathcal{X} = \mathbb{N} \setminus \{0, 1\}$ .

Let  $\mathcal{R} = \{x_1 + x_2 \rightarrow x_1 + x_3 : x_1, x_2, x_3 \in \mathcal{X} \wedge x_1 < x_2 \wedge x_3 = x_2/x_1\}$

Let  $A(X) \sim \mathcal{A}(X, \mathcal{R})$ .

Let  $X_0 \sim \wp^*(\mathcal{X})$ .

# Experiment Time!

**Example 2** ([2]). A soup of natural numbers divides each other...?

Let  $\mathcal{X} = \mathbb{N} \setminus \{0, 1\}$ .

Let  $\mathcal{R} = \{x_1 + x_2 \rightarrow x_1 + x_3 : x_1, x_2, x_3 \in \mathcal{X} \wedge x_1 < x_2 \wedge x_3 = x_2/x_1\}$

Let  $A(X) \sim \mathcal{A}(X, \mathcal{R})$ .

Let  $X_0 \sim \wp^*(\mathcal{X})$ .

results?



**Example 3.** A soup of single-digit numbers multiplies each other...?

Let  $\mathcal{X} = \{0, 1, 2, 3, 4, 5, 6, 7, 8, 9\}$ .

Let  $\mathcal{R} = \{x_1 + x_2 \rightarrow x_3 + x_3 : x_1, x_2, x_3 \in \mathcal{X} \wedge x_3 = x_1 \cdot x_2 \bmod 10\}$

Let  $A(X) \sim \mathcal{A}(X, \mathcal{R})$ .

Let  $X_0 \sim \wp^*(\mathcal{X})$ .

# Experiment Time!

**Example 3.** A soup of single-digit numbers multiplies each other...?

Let  $\mathcal{X} = \{0, 1, 2, 3, 4, 5, 6, 7, 8, 9\}$ .

Let  $\mathcal{R} = \{x_1 + x_2 \rightarrow x_3 + x_3 : x_1, x_2, x_3 \in \mathcal{X} \wedge x_3 = x_1 \cdot x_2 \bmod 10\}$

Let  $A(X) \sim \mathcal{A}(X, \mathcal{R})$ .

Let  $X_0 \sim \wp^*(\mathcal{X})$ .

results?

# Stirring Soup

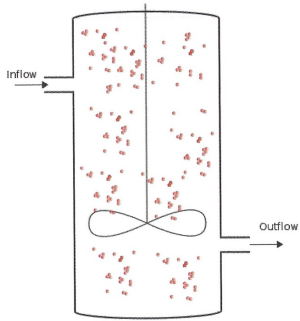


Figure 7.3 Well-stirred flow reactor with dilution — inflow and outflow of material.

# Stochasticity

# No-Ops, Creation, and Destruction

**Example 4.** A soup of functions writing into each other...?

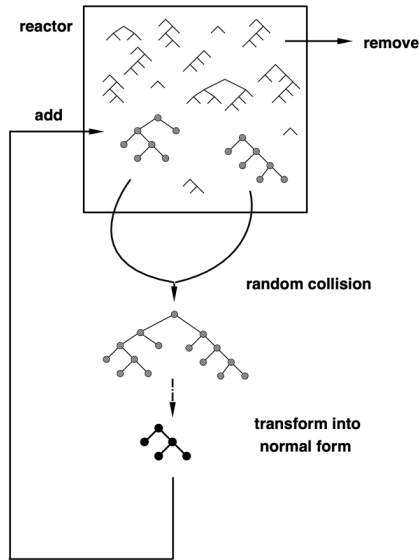
Let  $\mathcal{X} = \mathcal{X} \rightarrow \mathcal{X}$ , i.e., the space of all functions that can take functions and return new functions.

Let  $\mathcal{R} = \{x_1 + x_2 \rightarrow x_1 + x_3 \mid x_1, x_2, x_3 \in \mathcal{X} \wedge x_3 = x_1(x_2)\}$ , i.e., when two functions  $x_1, x_2$  meet,  $x_2$  is overwritten with the result of applying  $x_1$  to  $x_2$ .

Let  $A(X) \sim \mathcal{A}(X, \mathcal{R})$ .

Let  $X_0 \sim \wp^*(\mathcal{X})$ .

# Example: $\lambda$ -calculus



**Figure 1:**  $\lambda$ -calculus flow reactor. Two expressions  $A$  and  $B$  are chosen at random and a new object,  $(A)B$ , is constructed by “application” (see appendix A.2). Putting  $(A)B$  into its normal form by  $\beta$ -reduction (see appendix A.2), effectively decides which object species (i.e., “stable molecular formula”) the new object is an instance of.



# On $\lambda$ -calculus...

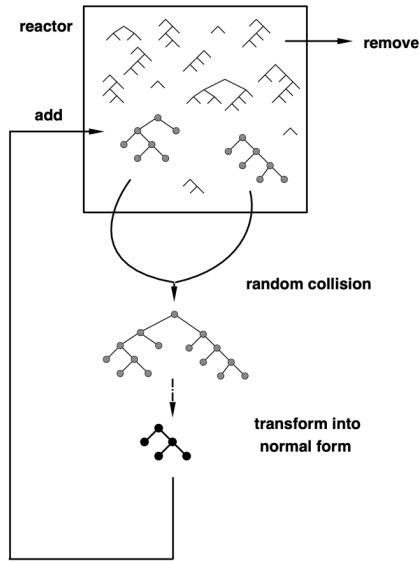
Example:

$$(\lambda x. f \ x) \ y \quad \longrightarrow \quad f \ y$$

Example in Python:

$$(\text{lambda } x: \ f(x))(y) \quad \longrightarrow \quad f(y)$$

# Example: $\lambda$ -calculus



**Figure 1:**  $\lambda$ -calculus flow reactor. Two expressions  $A$  and  $B$  are chosen at random and a new object,  $(A)B$ , is constructed by “application” (see appendix A.2). Putting  $(A)B$  into its normal form by  $\beta$ -reduction (see appendix A.2), effectively decides which object species (i.e., “stable molecular formula”) the new object is an instance of.

# Experiment Time??

**Example 4.** A soup of functions writing into each other...?

Let  $\mathcal{X} = \mathcal{X} \rightarrow \mathcal{X}$ , i.e., the space of all functions that can take functions and return new functions.

Let  $\mathcal{R} = \{x_1 + x_2 \rightarrow x_1 + x_3 \mid x_1, x_2, x_3 \in \mathcal{X} \wedge x_3 = x_1(x_2)\}$ , i.e., when two functions  $x_1, x_2$  meet,  $x_2$  is overwritten with the result of applying  $x_1$  to  $x_2$ .

Let  $A(X) \sim \mathcal{A}(X, \mathcal{R})$ .

Let  $X_0 \sim \wp^*(\mathcal{X})$ .

# Stability

# Mere Stability vs. Self-Replication

# Example: $\lambda$ -calculus

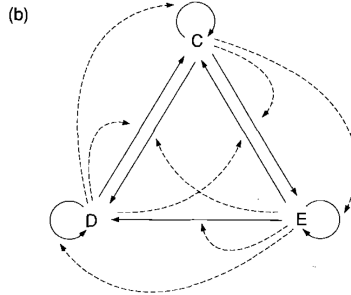
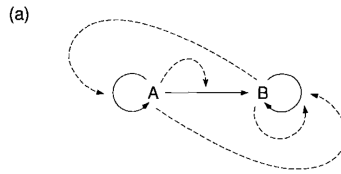


Figure 1. Level 0 ecology. A solid arrow denotes a transformation from argument object to value object. A dotted arrow connects the operator object with the transformation performed by it. Thus,  $E \circ C = D$  is shown as a solid arrow from C to D pointed at by a dotted arrow originating in E.

$$\begin{aligned}
 A &\equiv \lambda x_1 . (x_1) \lambda x_2 . \lambda x_3 . (x_3) \lambda x_4 . \lambda x_5 . (x_5) x_4, & B &\equiv \lambda x_1 . (x_1) \lambda x_2 . \lambda x_3 . (x_3) x_2, \\
 C &\equiv \lambda x_1 . \lambda x_2 . (x_1) \lambda x_3 . x_1, & D &\equiv \lambda x_1 . \lambda x_2 . \lambda x_3 . (x_2) \lambda x_4 . x_2, \\
 E &\equiv \lambda x_1 . \lambda x_2 . \lambda x_3 . \lambda x_4 . (x_3) \lambda x_5 . x_3.
 \end{aligned}$$

# Properties of Self-Replicators

longevity

fecundity

copy-fidelity

# Levels of Organization

Level 0: reproduction and ecology

- cycle of replicators (easily collapses to only one replicator)

Level 1: self-maintenance and organization

- emerges when mere copy actions are forbidden
- structure and interaction between particles can be described with specific rules
- part of the organization can generate all its elements anew

Level 2: organization of organizations

- create L1 organizations independently and then put them into the same soup
- organizations maintain themselves separately but interact via “glue” particles



# Emergence of Implicit Target Functions

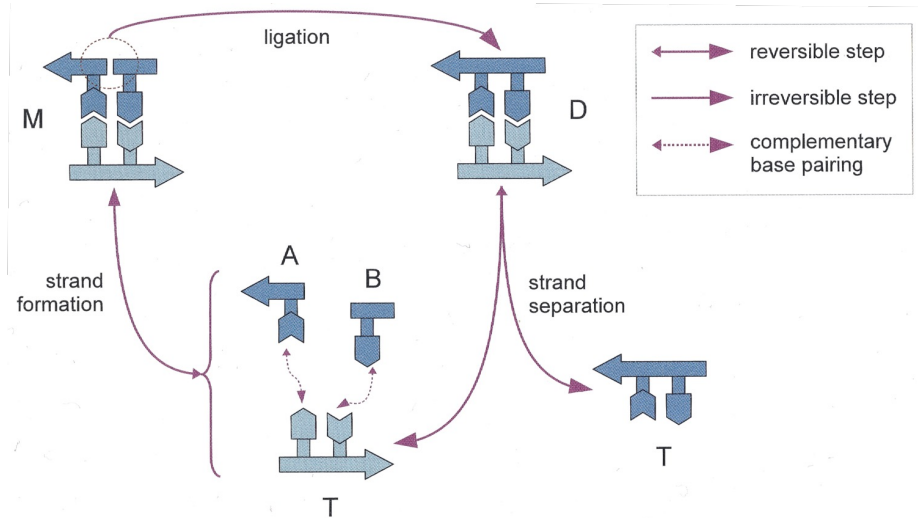


Figure 6.6 A model of the minimal self-replicator demonstrated by Von Kiedrowski [890]: The two *T* molecules are complementary templates (hexamers in the experiments). Each of them base-pairs with units *A* and *B* (trimers in the experiments), to form an intermediate molecule *M* which is almost a double strand: the still missing link between *A* and *B* is then catalyzed by *T* itself, forming the double strand *D*. *D* then splits into two single strands *T* and the cycle repeats. Adapted from [890].

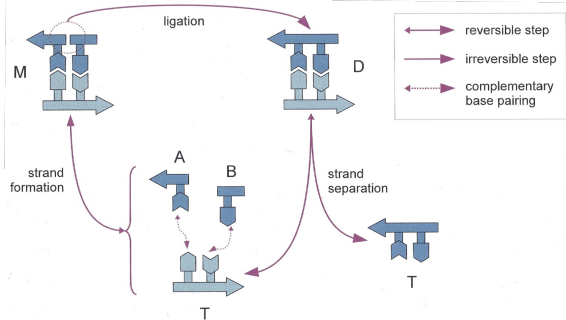


Figure 6.6 A model of the minimal self-replicator demonstrated by Von Kiedrowski [890]: The two  $T$  molecules are complementary templates (hexamers in the experiments). Each of them base-pairs with units  $A$  and  $B$  (trimers in the experiments), to form an intermediate molecule  $M$  which is almost a double strand: the still missing link between  $A$  and  $B$  is then catalyzed by  $T$  itself, forming the double strand  $D$ .  $D$  then splits into two single strands  $T$  and the cycle repeats. Adapted from [890].

**Example 5.** A DNA soup.

Let  $\mathcal{X} = \{A, C, G, T\}^{2 \times 2} \cup \{A, C, G, T\}^2 \cup \{A, C, G, T\}^1$ .

Let  $\mathcal{R} = \{R_{1,-,-}, R_{2,-,-}, R_{3,-,-}, R_{4,-,-}\}$  with

$$\begin{aligned}
 R_{1,X,Y} &= (X \ Y) + (-X) + (-Y) && \xrightarrow{0.5} \begin{pmatrix} X & Y \\ -X & -Y \end{pmatrix} \\
 R_{2,X,Y} &= \begin{pmatrix} X & Y \\ -X & -Y \end{pmatrix} && \xrightarrow{\quad} (X \ Y) + (-Y \ -X) \\
 R_{3,X,Y} &= (X) + (Y) && \xrightleftharpoons[0.01]{0.01} (X \ Y) \\
 R_{4,X} &= \emptyset && \xrightarrow{0.1} (X)
 \end{aligned}$$

for all  $X, Y \in \{A, C, G, T\}$ .

Let  $A(X) \sim \mathcal{A}(X, \mathcal{R})$ .

Let  $X_0 = \emptyset$ .